Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Abstract

Multi-object trackers rely on score thresholds tuned for one detector. When the detector is replaced, retrained, or compressed, its scores shift, and the tuned thresholds silently stop fitting. This paper proposes a training-free self-calibrating control layer (SCL) between an unchanged detector and tracker. SCL splits the logits of recent scores twice with Otsu's method and classifies the stream as clean or noisy, without labels or future frames. Its novelty is selective, host-aware adaptation. On clean streams it keeps the tracker's scores and lowers a threshold only if it would reject the confident detections; on noisy streams it selects and rescores the detections and sets the thresholds. It acts through four published tracker settings and passes on weak detections that continue tracks, so one frozen configuration serves single-stage and two-stage trackers. It was evaluated on seven trackers, four detection sources, and three datasets; changes are relative to the tracker alone. When detector scores were halved, two trackers fell to zero higher order tracking accuracy (HOTA); with SCL both recovered at least 97% of it. With a replacement detector, RT-DETR-L, SCL raised the multiple object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 12.6% and 16.3% and cut their identity switches by 43% and 47%, respectively. It also raised the HOTA of PD-SORT, unseen during design, by 0.9%, and kept four well-matched trackers within 0.1%. SCL adds on average no more than 2.95 ms per frame on an ordinary processor, so a detector can be replaced or retrained without retuning the tracker.

Keywords: multi-object tracking, tracking by detection, detector confidence calibration, online adaptation, Otsu thresholding, real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video and gives each one an identity number (ID) that stays the same over time, for surveillance, driving, and drone systems [1]. In the tracking-by-detection design [2], a detector returns candidate boxes with confidence scores, and a tracker links them into tracks [3–5], using a box only if its score passes a fixed threshold. A two-stage tracker, such as ByteTrack, also uses lower-score boxes in a second step to continue tracks [4]; a single-stage tracker, such as OC-SORT, ignores every box below one threshold [5].

In real systems the detector is often replaced, retrained, or compressed, while the tracker keeps thresholds set for another detector. Detector scores are not calibrated across models [6], so a new detector usually gives scores on a different scale. ByteTrack in its official setting starts a track only from a box with a score of at least 0.7; when the scores are halved, a car scored 0.9 gets 0.45, no track starts, and the higher order tracking accuracy (HOTA) [7] falls from 67.70 to 0 (Sect. 5.3). Retuning every tracker for every detector needs labels, adaptive-threshold methods are each built into one tracker [8–10], score calibration needs labelled data [6], and adaptive inference methods learn their settings for one model [11].

This paper presents a self-calibrating control layer (SCL) between a detector and a tracker that are both kept fixed (frozen): their code, weights, and stored settings are unchanged. We call the tracker that SCL serves the *host*. The novelty of SCL is *selective, host-aware adaptation*. It is selective because it first decides, from the detector scores alone, whether the stream fits the host: when most higher scores of recent frames form a clearly confident group, the stream is clean and

the host keeps its scores and thresholds; only when many scores are uncertain does SCL choose the candidates, give them new scores, and set the thresholds. It is host-aware because every action is expressed through four published settings of the host, the host contract (Sect. 3.7), so the same frozen rules serve single-stage and two-stage trackers. With halved scores, SCL restored ByteTrack and OC-SORT from 0 to about 66 HOTA; with other detectors, it raised the multiple object tracking accuracy (MOTA) of two-stage trackers by up to 24.7 points and cut their identity switches by 43% to 56%, while well-matched strong trackers kept their own output. The main contributions are:

1. *Selective adaptation:* a label-free, causal compatibility test (nested Otsu splits of recent logits and a whole-stream majority rule) that leaves a well-matched system as it is and corrects a mismatched one (Sects. 3.2–3.4).
2. *Host-aware adaptation:* a host contract of four published values through which one fixed configuration serves unchanged single-stage and two-stage trackers, with a track-guided recovery of weak detections (Sects. 3.5–3.7).
3. *Evidence:* one frozen configuration evaluated on seven trackers, four detection sources, and three datasets, with paired bootstrap intervals, adaptive-threshold baselines, the published ACT method, an ablation, and a sensitivity analysis (Sect. 5), at about 3 ms per frame on a central processing unit (CPU) (Sect. 6).

Section 2 presents the datasets, evaluation metrics, and related work. Section 3 describes the proposed SCL framework, while Section 4 details its development and refinement. Section 5 presents the experimental results and evaluation. Finally, Sections 6 and 7 discuss the practical implications and summarize the main findings and future research directions, respectively.

2 Background and Related Work

2.1 Datasets

MOT17 [12] contains people filmed with static and moving cameras; as is common, the second half of its seven training sequences, val-half (2,652 frames), is used for evaluation [4], with the published YOLOX-X detections [13] at the two published emission floors (lowest emitted scores) of 0.01 and 0.1. To study score changes directly, these scores were also changed before the tracker by three order-preserving transforms: cubing (s^3), halving ($0.5\,s$), and logit temperature $T = 2$, which divides the logit of each score (Sect. 3.1) by T. *KITTI tracking* [14] has 21 training sequences (8,008 frames) recorded from a car, scored on cars and pedestrians, and is used with two other detectors, YOLOv8n [15] and RT-DETR-L [16]. *UAVDT* [17] contains drone videos of vehicles; its 20 test sequences (16,592 frames) are used with ByteTrack, YOLOv8n, and RT-DETR-L. MOT17 with four trackers, KITTI, and the score transforms were development data. Two predeclared external trackers, PD-SORT and Hybrid-SORT, were run once with the frozen configuration on their authors' MOT17 val-half detections, and the held-out UAVDT test split was run once after a written plan. Each external host was checked against its authors' number: EXACT within 0.2 HOTA points, CLOSE within 1.0.

2.2 Evaluation metrics

HOTA [7], the main metric, balances finding the objects and keeping their identities,

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha \, \mathrm{AssA}_\alpha}, \tag{1}$$

where DetA_α and AssA_α are the detection and association accuracy at the localization threshold $\alpha = 0.05, 0.10, \ldots, 0.95$.

MOTA [18] and IDF1 [19] are

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{2}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}, \tag{3}$$

where FN are missed objects (false negatives), FP false positives, IDS identity switches, and GT the number of ground-truth boxes; IDTP, IDFP, and IDFN count boxes after each ground-truth track is matched to at most one output track.

All metrics are computed with TrackEval (commit 12c8791) and the official ground truth [20]; KITTI HOTA is averaged over cars and

pedestrians, and on UAVDT all vehicles form one class and predictions in ignored regions are removed. HOTA, MOTA, and IDF1 are in percent and higher is better (↑); IDS is a count and lower is better (↓); differences are in points. Each tracker runs alone (Host) and with SCL (Host+SCL) on the same sequences and detections, and Change is Host+SCL minus Host. Paired bootstrap resampling of whole sequences [21], 10,000 times with seed 42, gives a 95% interval for each change, which is significant (bold in the tables) when its interval does not include zero.

2.3 Related work

Tracking by detection. SORT matches Kalman-filter predictions to new detections by their overlap [3]; ByteTrack added a second step for low-score detections [4]; OC-SORT uses one score threshold [5]. Their thresholds were tuned for one detector, usually YOLOX [13]. Many recent trackers improve association with trained or domain-specific parts [22–27], which change or retrain the tracker; SCL keeps both unchanged. Seven published trackers serve as hosts with the authors' code and configuration: ByteTrack, BoT-SORT [28], OC-SORT, SparseTrack [29], BoostTrack [30], PD-SORT [31], and Hybrid-SORT [32]. ByteTrack is used with its authors' MOT17 thresholds (official setting) and with the Ultralytics defaults, which also serve BoT-SORT (library setting).

Adaptive thresholds. Adaptive ByteTrack sets ByteTrack's threshold from each frame [8], AdapTrack adapts its matching thresholds [9], and ACT sets the high and low thresholds from the first quartile and the number of the current scores [10]. ConfTrack weights the Kalman update and the matching cost by the score [33], and PlugTrack blends motion predictors with a trained network [34].

Comparison. Table 1 compares SCL with these methods. The adaptive-thresholding methods act in every frame, read only the current frame or each box, and are built into one tracker; calibration needs labelled data. SCL first decides from past scores whether to act at all, serves unchanged single-stage and two-stage trackers through their published settings, and needs neither labels nor trained parts (Sect. 3.4).

Score calibration and adaptive inference. Detector families such as YOLOX, YOLOv8, and RT-DETR give different score distributions, and calibration methods learn a mapping from labelled data [6]. Otsu's method picks, without labels, the threshold that best splits a histogram into two groups [35]; SCL applies it to the logits of recent scores. Dynamic networks learn their settings for each model [11], and an earlier controller, ACMOT (under review), sets the detector operating point from image and track cues. SCL instead reads the scores and acts on the tracker's input and thresholds.

3 Proposed SCL Framework

SCL is a lightweight, rule-based layer operating between a frozen detector and tracker (the host) without modifying their underlying algorithms. Fig. 1 illustrates the five processing steps of SCL for frame t. First, Otsu-based logit partitioning (Step 1) divides recent detection scores into background, uncertain, and confident groups. Next, detection stream classification (Step 2) determines whether the score distribution is clean or noisy based on historical confidence statistics. Mode-adaptive candidate selection (Step 3) then retains, removes, or remaps detection scores and adjusts the tracker's confidence thresholds according to the operating mode. Track-guided weak detection recovery (Step 4) allows eligible low-confidence detections to support track continuity. Finally, camera motion-adaptive matching tolerance (Step 5) adjusts the association criterion according to estimated camera motion. The host contract provides the tracker's published parameters, while the tracker performs its original association and track-management operations unchanged.

Table 2 lists every symbol in plain words, with its value in the clean stream of Fig. 2a, which the subsections below follow from input to output.

3.1 Inputs and outputs of SCL

In each frame t, the detector gives candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$, each with a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class c_i. The layer works on the logit

$$z_i = \log \frac{s_i}{1 - s_i}, \tag{4}$$

which keeps the order of the scores (0.5 gives $z_i = 0$, 0.9 gives $\log 9 = 2.2$, and 0.1 gives

Table 1 SCL compared with adaptive-thresholding and score-calibration methods: what each method changes, the information it uses, whether it needs labels or trained parts, when it acts, and where it works. ReID: re-identification network.

Method	What is adapted	Information used	Labels or trained parts	When it acts	Where it works
Adaptive thresholding					
Adaptive ByteTrack [8]	high/low score split	scores of the current frame	none	every frame	inside ByteTrack
AdapTrack [9]	matching thresholds	current frame	ReID network	every frame	its own tracker
ACT [10]	high and low thresholds, from preset levels	first quartile and number of the current scores	none	every frame	inside the tracker
ConfTrack [33]	Kalman update, matching cost	score of each box	ReID network	every box	its own tracker
Score calibration					
Küppers et al. [6]	scores mapped to probabilities	labelled data	calibration map	every score	detector output
This paper					
SCL	candidates, scores, thresholds, matching tolerance	logits of past frames; host contract	none	noisy streams (clean: capped thresholds)	between a detector and unchanged trackers

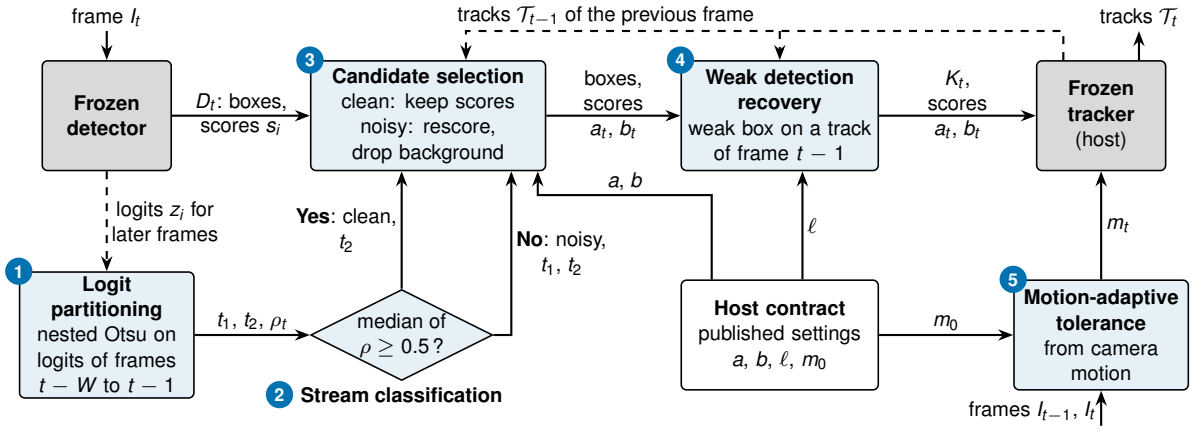


Fig. 1 SCL between a frozen detector and a frozen tracker (the host), for frame t. Arrows are labelled with the data they carry; dashed arrows carry data from earlier frames. The diamond (Step 2) is a decision: clean (Yes) if the median of ρ is at least 0.5, noisy (No) otherwise. Steps 1–5: Sects. 3.2–3.6; host contract: Sect. 3.7; symbols: Table 2

Table 2 The symbols of Sect. 3 in plain words. Example: the value in the clean stream of Fig. 2a (MOT17-02, frame 101, ByteTrack in its official setting), as recorded by the frozen SCL; e.g.: a worked example from the text.

Symbol	Meaning	Example
s_i, z_i	detector score of candidate i, between 0 and 1, and its logit, Eq. (4)	e.g. 0.9 and $\log 9 = 2.2$
W, Z_t^W	number of past frames whose logits are pooled; the logits of frames $t-W$ to $t-1$	10; 916 logits
t_1, t_2	first split (background lies below it); second split (confident lies above it)	-0.78 (0.32); 0.95 (0.72)
ρ_t	share of confident candidates in the foreground, Eq. (6)	$175/263 = 0.67$
median ρ	median of ρ over the stream so far; clean if ≥ 0.5	0.63: clean
a, b, ℓ	the tracker's published scores to match a track and to start one, and the lowest score it uses	0.6, 0.7, 0.1
m_0	the tracker's matching tolerance: 1 minus the smallest IoU that matches	0.8 (IoU ≥ 0.2)
a_t, b_t, m_t	the thresholds and tolerance SCL passes for frame t, Eqs. (7) and (9)	0.6, 0.7 (kept); 0.8
u_i, $\tilde{s}_i$	rank of s_i among stored scores; the new score of the noisy mode, Eq. (8)	e.g. $u_i = 0.8 \to \tilde{s}_i = 0.9$
$\mathcal{T}_{t-1}$, K_t	the tracks of the previous frame; the candidates passed to the tracker	–
g_t, r_t	camera shift between two frames; its ratio to the usual shift	e.g. $r_t = 2$

-2.2) and spreads out the regions near 0 and 1, where detector scores crowd; the logistic function $\sigma(z) = 1/(1+e^{-z})$ maps a logit back to a score. The host gives its contract $h = (a, b, \ell, m_0)$: the score a needed to match a track, the score b needed to start one, the lowest score ℓ it uses, and

its matching tolerance m_0, one minus the smallest intersection over union (IoU) that counts as a match. A single-stage tracker has $\ell = \min(a, b)$, a two-stage tracker $\ell < \min(a, b)$. For ByteTrack in its official setting, $a = 0.6$, $b = 0.7$, $\ell = 0.1$, and $m_0 = 0.8$ (Table 3). In every frame the layer returns

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t;\, h\right): \quad (5)$$

the candidates K_t, their scores, and the thresholds and tolerance for the tracker, from the logits $Z_{<t}$ of earlier frames, the tracks $\mathcal{T}_{t-1}$, and the camera motion g_t between frames I_{t-1} and I_t; no future frame is used.

3.2 Otsu-Based Logit Partitioning and Confidence Estimation

Step 1 in Fig. 1. Input: the logits Z_t^W of the candidates of the last $W = 10$ frames. *Output:* the splits t_1, t_2 and the confident share ρ_t.

The logits are split twice with Otsu's method [35], which picks from the data alone the threshold that best separates two groups: t_1 separates background (low logits) from foreground, and t_2, found on the foreground only, divides it into uncertain and confident candidates. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{z \in Z_t^W : z \geq t_2\}\right|}{\left|\{z \in Z_t^W : z \geq t_1\}\right|}, \quad (6)$$

describes the shape of the score distribution. In the clean stream of Fig. 2a, the window holds 916 candidates: 653 below $t_1 = -0.78$ (score 0.32), 88 between the splits, and 175 above $t_2 = 0.95$ (score 0.72), so $\rho_t = 175/(88 + 175) = 0.67$; in the noisy stream of Fig. 2b, 1,037 are background, 99 uncertain, and 83 confident, so $\rho_t = 83/(99+83) = 0.46$. As clutter usually gives more candidates than objects do, a window with fewer candidates below t_1 than above it counts as clean, $\rho_t = 1$ (background check), which does not apply in Fig. 2.

3.3 Median-Based Detection Stream Classification

Step 2 in Fig. 1. Input: the values ρ of all frames so far. *Output:* the mode of frame t.

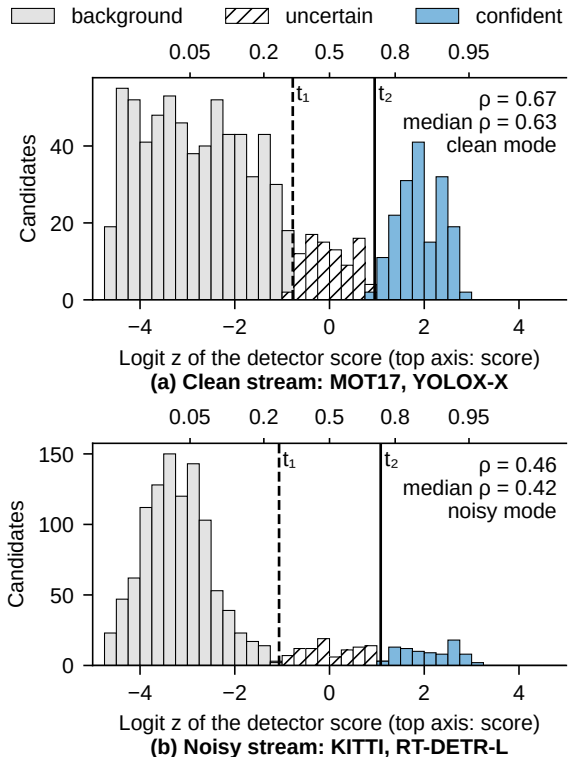


Fig. 2 What SCL sees before its decision for frame 101: the logits of the candidates of the last 10 frames (top axis: the same position as a score). Dashed line: t_1; solid line: t_2; light gray: background; hatched: uncertain; blue: confident. (a) MOT17 sequence MOT17-02, YOLOX-X, ByteTrack (official): median $\rho = 0.63 \geq 0.5$, clean. (b) KITTI sequence 0001, RT-DETR-L, ByteTrack (library): median $\rho = 0.42 < 0.5$, noisy. Values recorded by the frozen SCL

The mode describes the distribution of the scores, not whether a single detection is correct: the stream is clean at frame t if the median of all ρ values so far is at least one half, a majority rule, and noisy otherwise; before the first valid split it is cold. In Fig. 2a, frames 2 to 101 give a median of 0.63, so frame 101 is clean; in Fig. 2b the median is 0.42, so it is noisy. The whole-stream median keeps the mode stable against a few frames of blur or occlusion, while the groups, and in the clean mode the thresholds of Step 3, follow a new score scale within W frames.

Table 3 Host contracts: the four published settings (a, b, ℓ, m_0) of each tracker that SCL reads (Sect. 3.1), and where each value comes from; θ: the published IoU threshold, $m_0 = 1 - \theta$.

Tracker (setting)	Type	a	b	ℓ	m_0	Published settings behind the values
ByteTrack (official)	two-stage	0.6	0.7	0.1	0.8	track_thresh 0.6; new tracks +0.1; second step from 0.1; match_thresh 0.8
ByteTrack (library)	two-stage	0.25	0.25	0.1	0.8	Ultralytics bytetrack.yaml: high 0.25, new 0.25, low 0.1, match 0.8
BoT-SORT (library)	two-stage	0.25	0.25	0.1	0.8	Ultralytics botsort.yaml: the same four values
SparseTrack	two-stage	0.6	0.7	0.1	0.85	track_thresh 0.6; new tracks +0.1; second step from 0.1; match_thresh 0.85
BoostTrack	two-stage	0.6	0.6	0.1	0.7	det_thresh 0.6; every detection from 0.1 can be boosted; $\theta = 0.3$
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	det_thresh 0.6; low-score step from 0.1; $\theta = 0.25$
OC-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; $\theta = 0.3$
PD-SORT	single-stage	0.6	0.6	0.6	0.7	det_thresh 0.6, no low-score step; $\theta = 0.3$

3.4 Mode-Adaptive Detection Candidate Selection and Threshold Adjustment

Step 3 in Fig. 1. Input: D_t, the mode, t_1, t_2, the thresholds a, b, and the tracks $\mathcal{T}_{t-1}$. *Output:* the kept boxes, their scores, and the thresholds a_t, b_t.

Cold and clean modes. The tracker keeps its own scores, and only cross-class duplicates are removed. In the clean mode, a threshold that would reject the confident group is lowered to the score $\sigma(t_2)$ at which that group begins:

$$a_t = \min(a, \sigma(t_2)), \qquad b_t = \min(b, \sigma(t_2)). \quad (7)$$

In Fig. 2a, $\sigma(t_2) = 0.72$ is above both $a = 0.6$ and $b = 0.7$, so the published thresholds are kept and the output equals that of the host alone (Sect. 5). When the scores are halved, no score is above 0.5 and ByteTrack in its official setting never starts a track; $\sigma(t_2)$ falls with the scores below 0.5, Eq. (7) lowers both thresholds to it, and the confident candidates reach the tracker again.

Noisy mode. Background candidates ($z < t_1$) are dropped. Each remaining score is replaced by its mid-rank $u_i \in (0, 1)$ among the scores of a remap memory, which stores the scores kept in the first frame and in every 10th frame, the last 20 of them (equal scores count half):

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \quad (8)$$

and both thresholds are set to $a_t = b_t = 0.5$. A confident candidate with $u_i = 0.8$ gets $0.5 + 0.5 \cdot 0.8 = 0.9$ and may start and continue tracks; an uncertain one with $u_i = 0.3$ gets $0.1 + 0.4 \cdot 0.3 = 0.22$ and may only continue tracks through a tracker's low-score step. A weaker candidate that overlaps a stronger one with IoU above one half is removed, unless a different track of the previous frame claims it, which keeps people who stand close together in a crowd.

Score remapping is not probability calibration. Calibration learns from labelled data the probability that a detection is correct [6]. The remap needs no labels: it keeps the order of the scores within each group and places the groups in fixed ranges, so that the tracker's own thresholds read the layer's decision (above 0.5: "may start a track"; 0.1 to 0.5: "may only continue one"), and it is unchanged by any order-preserving change of the scores.

3.5 Track-Guided Weak Detection Recovery

Step 4 in Fig. 1. Input: the output of Step 3, the tracks $\mathcal{T}_{t-1}$, and ℓ. *Output:* K_t, their scores, and a_t, b_t for the tracker.

A track can be lost when its object gets low scores for a few frames, above all in a single-stage tracker, which has no low-score step. The rescue passes such a candidate, with its score raised just above the lowest threshold of the frame (ℓ for a two-stage tracker, the lower of a_t and b_t for a single-stage one), when it is in the foreground (or, for a single-stage tracker, when there is no background group), overlaps a track of the previous frame with IoU of at least one half, no other usable candidate covers that track, and the tracker could not see it otherwise. For ByteTrack in its official setting, a rescued candidate gets 0.101 and enters its low-score step; for OC-SORT in a noisy frame it gets 0.501. The tracker itself still decides, with its own matching, whether and to which track the box is assigned.

3.6 Camera Motion-Adaptive Matching Tolerance

Step 5 in Fig. 1. Input: the frames I_{t-1}, I_t and the tolerance m_0. *Output:* the tolerance m_t of frame t.

In noisy frames, the matching tolerance grows with camera motion, which moves boxes far from their tracks; otherwise $m_t = m_0$. The motion g_t is the shift between consecutive grayscale frames at quarter resolution, found by phase correlation [36], and r_t is its ratio to its median over the last 100 frames ($r_t = 1$ during a warm-up of 5 frames or if this median is zero):

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \qquad (9)$$

With $m_0 = 0.8$, a match needs an IoU of at least 0.2; if the camera moves twice as much as usual ($r_t = 2$), then $m_t = \min(0.95, 1 - 0.2/2) = 0.9$, and an IoU of 0.1 is enough. The cap 0.95 keeps a smallest overlap of 0.05.

3.7 SCL Integration with Existing Trackers

Each contract value in Table 3 comes from the tracker's published configuration; none is fitted. For each tracker family, a small adapter writes the layer's outputs into the tracker's existing parameters and calls its own update, so the authors' matching, motion model, track management, and post-processing run unchanged.

3.8 Fixed SCL Parameters

Table 4 lists all values of SCL. They were fixed before the external evaluation, and none was chosen by searching over a range of values: they are standard values (the IoU of 0.5 of MOT evaluation and the common thresholds 0.5 and 0.1), definitions, or design choices kept fixed in all development stages. Algorithm 1 gives Steps 1–5 for one frame.

Table 4 The fixed values of SCL, their settings, and their sources; the same values are used for every tracker, detector, and dataset.

Value	Setting	Source
Window W	10 frames	design choice
Mode rule	clean if median $\rho \geq 0.5$	majority rule
Background check	$\rho = 1$ if fewer below t_1	no background
Noisy score ranges	$[0.5, 1)$, $[0.1, 0.5)$	common values
Noisy a_t, b_t	0.5	common value
Remap memory	20 frames, every 10th	design choice
Duplicate, rescue IoU	0.5	MOT evaluation
Rescued score	lowest threshold $+0.001$	just above it
Motion cue	phase correlation, $\frac{1}{4}$ size	standard [36]
Motion history	100 frames, warm-up 5	design choice
Tolerance cap	0.95	overlap ≥ 0.05

4 SCL Development and Refinement

This section presents the development and refinement of SCL using the development data described in Sect. 2.1. Section 4.1 introduces the initial always-on banding (AOB) design and discusses its limitations (Table 5). Section 4.2 describes the incremental improvements that addressed these limitations, supported by the experimental comparisons in Table 6. Finally, Sect. 4.3 explains the configuration freezing and validation procedures performed before external evaluation to ensure reproducibility and prevent changes based on subsequent results.

4.1 Initial Design and Limitations

The earlier design AOB computed the same nested Otsu groups but treated every stream as noisy: in every frame it applied the groups, removed every weaker candidate that overlapped a stronger one with IoU above one half, and rescored all candidates by rank. On strong MOT17 trackers (Table 5), AOB lowered HOTA by 4.15 to 5.89 points, as the overlap rule deleted people hidden behind others in crowds and its thresholds were stricter than the trackers' own; with the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left BoostTrack, online and with its gap-filling post-processing (GBI), identical to the host. The next steps therefore let the layer decide from the scores whether to act at all.

4.2 Incremental Design Improvements

Each step was compared with the step before it (Table 6). The *mode gate* added the mode check, the clean mode that keeps the tracker's scores and thresholds, and the crowd-safe duplicate rule, and restored SparseTrack and BoostTrack to the

Table 5 MOT17 val-half, published YOLOX-X detections: the earlier design AOB and the final SCL, each added to the same strong tracker (Host). Change: the run with the layer minus Host, for AOB with its 95% bootstrap interval; bold: the interval does not include zero (a reliable change).

Metric	Host	Host+AOB	AOB change [95% CI]	Host+SCL	SCL change
SparseTrack [29]					
HOTA↑	68.88	64.72	**−4.15 [−5.54, −1.66]**	68.93	+0.06
MOTA↑	77.85	71.71	**−6.14 [−8.10, −2.74]**	77.93	+0.08
IDF1↑	81.97	77.49	**−4.49 [−6.19, −1.96]**	82.13	+0.16
BoostTrack, online [30]					
HOTA↑	68.49	62.61	**−5.89 [−7.76, −2.87]**	68.49	0 (identical)
MOTA↑	75.50	66.64	**−8.87 [−12.37, −7.14]**	75.50	0 (identical)
IDF1↑	81.41	75.19	**−6.22 [−9.33, −2.85]**	81.41	0 (identical)
BoostTrack, with GBI [30]					
HOTA↑	71.73	66.35	**−5.37 [−7.30, −2.75]**	71.73	0 (identical)
MOTA↑	81.03	72.30	**−8.73 [−12.59, −6.50]**	81.03	0 (identical)
IDF1↑	84.16	78.51	**−5.65 [−8.96, −2.77]**	84.16	0 (identical)

Table 6 How SCL was built from AOB on development data. Each row adds one change to the row above; Effect gives the HOTA (%) before and after that change ("A to B") or compared with the tracker alone (host). The number in parentheses after ByteTrack and OC-SORT is the lowest emitted detector score.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)

accuracy of their hosts. The *whole-stream mode* removed mode changes in the middle of streams and raised ByteTrack and OC-SORT, and the *continuation rescue* added 0.2 to 0.4 points for OC-SORT. The *background check* completed SCL: ByteTrack and BoostTrack returned to output identical to the host, and OC-SORT rose to 67.04 and 66.91 at the two floors.

4.3 Configuration Freezing and Validation

The final configuration was frozen before any external test as a version-control tag, with a lock file holding a cryptographic hash of the layer, its configuration, and the evaluation scripts; sixty-one automated tests check, among others, that the layer uses no future data and passes clean streams unchanged. All results of Sect. 5 come from this frozen configuration; the comparisons, ablation, and sensitivity analysis used run-time options of the evaluation scripts and changed none of its values.

5 Experimental Results and Evaluation

This section evaluates the effectiveness and generalization of SCL against the host trackers using identical detections and published tracker configurations. Section 5.1 examines tracking performance across different trackers on MOT17, while Sect. 5.2 evaluates detector–tracker compatibility on KITTI. Section 5.3 investigates robustness to detection confidence-score shifts, and Sect. 5.4 compares SCL with adaptive thresholding methods. Section 5.5 presents component ablation and parameter sensitivity analyses. Finally, Sects. 5.6 and 5.7 assess generalization to previously unseen trackers and the held-out UAVDT dataset, respectively. In all comparisons, the baseline is the host tracker operating without SCL, and Fig. 3 summarizes the HOTA differences across the evaluated configurations.

How to read the result tables. Each row gives one metric of the tracker alone (Host) and with

Algorithm 1 SCL for frame t; the comments name the steps of Fig. 1. The window holds the logits of frames $t-W$ to $t-1$, so the decision for frame t uses no future frame.

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table 4

Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i (Eq. (4))
if no valid split exists **then** (e.g., first frame)
    mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window (Step 1)
    $\rho_t \leftarrow$ Eq. (6) with the background check
    mode $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy (Step 2)
end if
if mode is cold or clean **then** (Step 3)
    remove cross-class duplicates; keep raw scores
    if clean, set (a_t, b_t) by Eq. (7)
else (noisy mode)
    remove duplicates with track context
    drop $z < t_1$; remap by Eq. (8); $a_t, b_t \leftarrow 0.5$
end if
rescue continuations (not cold) (Step 4)
$m_t \leftarrow$ Eq. (9) if noisy, else m_0 (Step 5)
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
update the window (logits left after duplicate removal), the motion history (g_t), and the stored tracks after the tracker update

SCL (Host+SCL), and Change with its 95% interval (Sect. 2.2); a bold change is reliable, and "identical" means an unchanged output. For example, in Table 7, OC-SORT at floor 0.01 has a MOTA of 74.67 alone and 75.94 with SCL: the change is +1.27 points, and its interval $[+0.17, +3.01]$ lies above zero, so the gain is reliable. Values are rounded separately, so Host+SCL minus Host can differ from Change in the last digit.

5.1 Performance Across Trackers on MOT17

Table 7 shows the development trackers on MOT17 val-half. The two-stage trackers with well-matched detections kept the accuracy of their hosts: BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave exactly the same output through the self-limiting clean mode. The single-stage OC-SORT improved at both floors (HOTA from 66.43 to 67.04 and from 66.44 to 66.91, MOTA +1.27 and +1.26), with 8 fewer identity switches at floor 0.01 and higher HOTA on 7 of 7 sequences, through the continuation rescue.

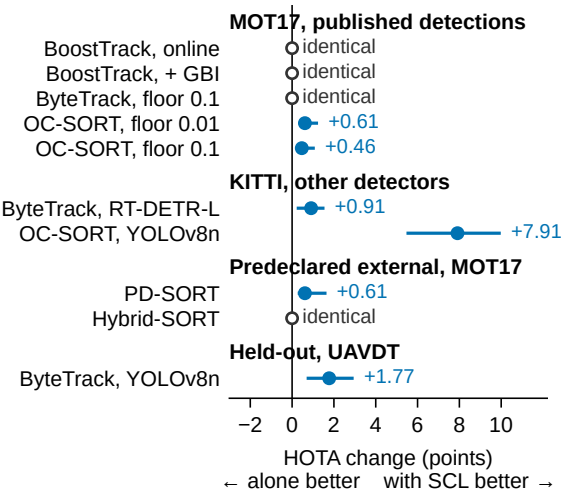


Fig. 3 HOTA change with SCL (tracker with SCL minus tracker alone, in points) on MOT17, KITTI, and UAVDT. Dot and number: the change; line: its 95% bootstrap interval; right of zero, SCL is better. Filled dot: a reliable gain; open dot: identical output

5.2 Cross-Detector Evaluation on KITTI

Table 8 shows what happens when the detector is replaced. With the noisy RT-DETR-L, SCL raised ByteTrack by 5.98 MOTA, 3.87 IDF1, and 0.91 HOTA points and BoT-SORT by 7.86 MOTA and 3.10 IDF1 points, with 297 and 226 fewer identity switches. With YOLOv8n, OC-SORT rose from 36.27 to 44.18 HOTA (+7.91 points, $[+5.47, +9.98]$), 9.40 MOTA points, and 9.60 IDF1 points, as its missed objects (FN) fell by 5,310. Fig. 4 follows this cell over three frames: each object keeps its ID, and at frame 102 OC-SORT alone tracked 6 of the 10 labelled objects and with SCL 9.

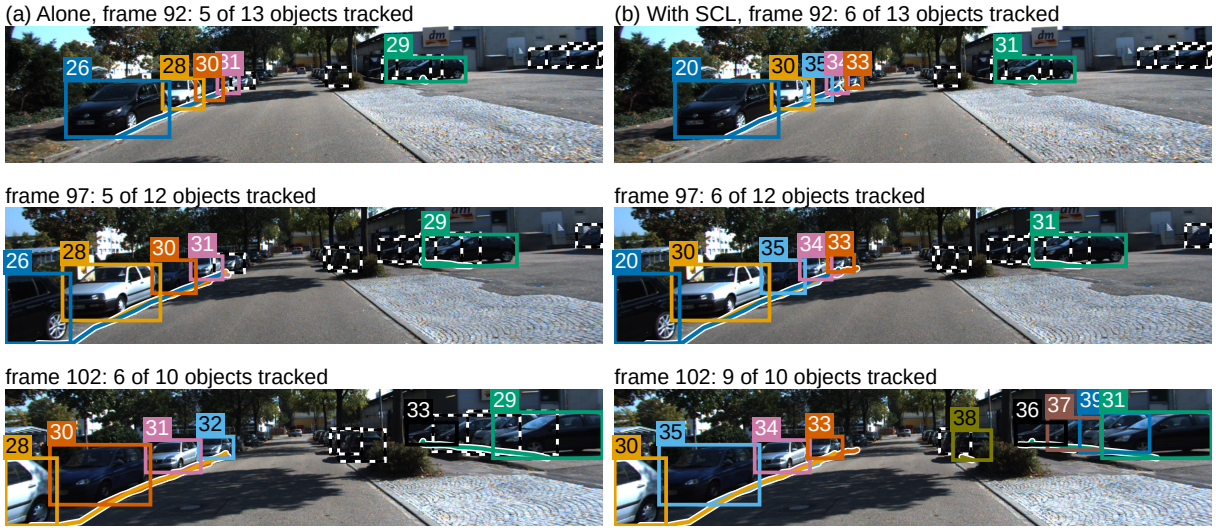


Fig. 4 KITTI sequence 0001 [14] tracked by OC-SORT with YOLOv8n at frames 92, 97, and 102 (rows): (a) the tracker alone; (b) with SCL. Each colour and number is one track; its line shows the object's last 20 frames. Dashed boxes: labelled objects that no track follows. IDs are numbered separately in each run. The sequence has the median HOTA gain of the 21 sequences, and frame 102 is its first frame with the largest gain in tracked objects

Table 7 MOT17 val-half, published YOLOX-X detections: each development tracker alone (Host) and with SCL (Host+SCL). Change: Host+SCL minus Host, with its 95% bootstrap interval; bold: a reliable change. Paper: the value reported by the tracker's authors (–: not reported). Floor: the lowest emitted detector score.

Metric	Paper	Host	Host +SCL	Change [95% CI]
BoostTrack, online [30]				
HOTA↑	68.37	68.49	68.49	0 (identical)
MOTA↑	75.56	75.50	75.50	0 (identical)
IDF1↑	81.35	81.41	81.41	0 (identical)
IDS↓	118	113	113	0 (identical)
BoostTrack, + GBI [30]				
HOTA↑	71.33	71.73	71.73	0 (identical)
MOTA↑	80.55	81.03	81.03	0 (identical)
IDF1↑	83.84	84.16	84.16	0 (identical)
IDS↓	106	97	97	0 (identical)
ByteTrack, floor 0.1 [4]				
HOTA↑	–	67.70	67.70	0 (identical)
MOTA↑	76.6	77.60	77.60	0 (identical)
IDF1↑	79.3	79.47	79.47	0 (identical)
IDS↓	–	214	214	0 (identical)
OC-SORT, floor 0.01 [5]				
HOTA↑	66.5	66.43	67.04	**+0.61** [**+0.36**, **+1.24**]
MOTA↑	74.9	74.67	75.94	**+1.27** [**+0.17**, **+3.01**]
IDS↓	–	211	203	**−8** [**−15**, **−2**]
OC-SORT, floor 0.1 [5]				
HOTA↑	66.5	66.44	66.91	**+0.46** [**+0.27**, **+1.09**]
MOTA↑	74.9	74.67	75.93	**+1.26** [**+0.37**, **+3.34**]
IDF1↑	77.7	78.05	78.33	**+0.287** [**+0.003**, **+0.838**]

Table 8 KITTI tracking training with detectors the trackers were not tuned for (HOTA averaged over cars and pedestrians); columns as in Table 7. BoT-SORT leaves out sequence 0020 in both runs, because its code stops with a numerical error on it.

Metric	Host	Host +SCL	Change [95% CI]
ByteTrack [4], *RT-DETR-L*			
HOTA↑	50.73	51.64	**+0.91** [**+0.22**, **+1.55**]
MOTA↑	47.27	53.24	**+5.98** [**+1.16**, **+9.94**]
IDF1↑	64.98	68.86	**+3.87** [**+1.67**, **+5.06**]
IDS↓	686	389	**−297** [**−530**, **−122**]
BoT-SORT [28], *RT-DETR-L* (20 sequences)			
MOTA↑	48.25	56.11	**+7.86** [**+2.21**, **+13.44**]
IDF1↑	67.74	70.85	**+3.10** [**+1.00**, **+4.57**]
IDS↓	476	250	**−226** [**−369**, **−120**]
OC-SORT [5], *YOLOv8n*			
HOTA↑	36.27	44.18	**+7.91** [**+5.47**, **+9.98**]
MOTA↑	33.99	43.39	**+9.40** [**+1.10**, **+12.30**]
IDF1↑	50.46	60.05	**+9.60** [**+5.61**, **+12.16**]

5.3 Robustness to Detection Score Shifts

Table 9 shows the trackers under the score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA; with the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60; BoostTrack rose from 61.33 to 64.84. With $T = 2$, the layer raised

ByteTrack and OC-SORT by 1.11 and 1.24 points. Across the score changes, the layer raised HOTA significantly in 7 conditions.

5.4 Comparison with Adaptive Thresholding Methods

Two simple baselines set the tracker's matching and birth thresholds in every frame to the first or second Otsu split of the logits of the last 10 frames, t_1 or t_2, and pass the scores unchanged. On KITTI (Table 10), SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the baselines, with intervals against the t_2 baseline above zero in every cell, so adaptive thresholds alone do not explain the gains.

Logit-to-score conversion. The splits are found on logits and converted to scores with the logistic function; the thresholds recorded in every frame equalled the intended values, $\sigma(t_1)$ or $\sigma(t_2)$ in all eight baseline runs, and $\min(a, \sigma(t_2))$ (Eq. (7)) or 0.5 in the four KITTI cells of SCL.

ACT. ACT [10], the closest published method (Sect. 2.3), was run with its published parameters on the same detections and hosts as SCL (Table 11): its high threshold became the host's matching threshold (with OC-SORT, its single threshold) and its low threshold the lower bound of the second step. With the host's own thresholds, the ACT runner reproduced the host exactly in all 84 KITTI and 40 UAVDT sequence runs.

Table 9 MOT17 val-half with the detector scores changed before the tracker, which keeps its published thresholds: s^3 cubes every score, $0.5\,s$ halves it, and $T = 2$ divides its logit by 2. HOTA of the tracker alone (Host) and with SCL (Host+SCL); 0.00: no object tracked; Change with its 95% bootstrap interval; bold: a reliable change. Lowest emitted score 0.01 (BoostTrack 0.1).

	HOTA (%)		
	Host		
Score change	Host	+SCL	Change [95% CI]
ByteTrack, official setting [4]			
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**
OC-SORT [5]			
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**
BoostTrack, online [30]			
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**
$T = 2$	67.59	67.59	0 (identical)

Table 10 KITTI tracking training: HOTA of the tracker alone (Host), of two adaptive baselines that set the tracker's thresholds in every frame to the first (t_1) or second (t_2) Otsu split of the logits of the last 10 frames, and of SCL. Halved: halved detector scores. SCL − t_2: the gain over the stronger baseline, with its 95% bootstrap interval below; bold: a reliable gain.

Cell	Host	t_1	t_2	SCL	SCL − t_2
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.18	**+3.09 [+2.00, +4.03]**
ByteTrack, RT-DETR-L, halved	49.50	49.58	49.50	51.79	**+2.29 [+0.28, +4.96]**
OC-SORT, YOLOv8n, halved	0.00	37.68	43.56	46.30	**+2.74 [+1.32, +3.34]**

SCL reached significantly higher HOTA than ACT in every row of Table 11a: +2.01 and +10.06 points under halved scores and +1.82 with YOLOv8n on UAVDT. On UAVDT with RT-DETR-L (Table 11b), SCL reached a MOTA of 27.99 against 5.34 for ACT (+22.65, [+9.03, +47.21]) and had 418 fewer identity switches, because the fixed score levels of ACT removed few of the host's false positives.

5.5 Component Ablation and Parameter Sensitivity

Table 12 removes one part of the frozen SCL, or changes one of its values, at a time on the three KITTI cells of Table 10. Each part matters where its step acts. Without the weak detection recovery, the single-stage OC-SORT lost up to 3.35 HOTA points, as it has no low-score step of its own; without the noisy mode (always clean), the halved-score cells lost up to 2.44 points, without the remap up to 1.67, and without the motion tolerance up to 0.14. The clean mode, in turn, keeps well-matched trackers at their own accuracy (Table 5).

The sensitivity analysis, planned before the runs, changed the window W, the frames whose median decides the mode, the IoU of the duplicate and recovery rules, and the motion history. For every value and cell, HOTA stayed within 0.62 points of the frozen SCL, and within 0.24 points on a fourth cell, ByteTrack with the original RT-DETR-L scores. None of the values thus needs tuning for these detectors and trackers.

Table 11 ACT [10], with its published parameters, and SCL on the same detections and tracker. (a) HOTA with each method, and SCL − ACT with its 95% bootstrap interval. (b) MOTA and identity switches on UAVDT with RT-DETR-L. A positive HOTA or MOTA difference and a negative IDS difference favour SCL; bold: a reliable difference. KITTI: HOTA averaged over cars and pedestrians; UAVDT: test split.

(a) HOTA

Data	Detector	Tracker	ACT	SCL	SCL − ACT [95% CI]
KITTI, halved	RT-DETR-L	ByteTrack	49.77	51.79	**+2.01 [+0.01, +4.76]**
KITTI, halved	YOLOv8n	OC-SORT	36.24	46.30	**+10.06 [+6.88, +12.01]**
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	**+1.82 [+0.65, +3.00]**

(b) MOTA and identity switches on UAVDT (test split, ByteTrack, RT-DETR-L)

	MOTA↑			IDS↓		
Detector	ACT	SCL	SCL − ACT [95% CI]	ACT	SCL	SCL − ACT [95% CI]
RT-DETR-L	5.34	27.99	**+22.65 [+9.03, +47.21]**	873	455	**−418 [−718, −123]**

Table 12 What each part and each fixed value of SCL contributes on the three KITTI cells of Table 10 (HOTA): (a) OC-SORT, YOLOv8n; (b) ByteTrack, RT-DETR-L, halved scores; (c) OC-SORT, YOLOv8n, halved scores. Middle: one part removed at a time, with the largest HOTA loss and its cell. Lower: one fixed value changed at a time (frozen value in brackets), with the largest HOTA change over the tested values and cells.

Frozen SCL, HOTA: (a) 44.18, (b) 51.79, (c) 46.30		
Part removed	Largest loss	Cell
Weak detection recovery	3.35	(a)
Noisy mode (always clean)	2.44	(c)
Remap (raw scores)	1.67	(b)
Motion tolerance	0.14	(c)
Value changed (frozen)	**Largest change**	**Tested**
Window W (10 frames)	0.56	5, 20
Mode median frames (all)	0.36	last 100, 300
Duplicate/recovery IoU (0.5)	0.62	0.4, 0.6
Motion history (100 frames)	0.09	50, 200

Table 13 Generalization, each run once with the frozen SCL; columns as in Table 7. (a) The two external trackers, chosen before the run and not used during design (MOT17 val-half, the authors' detections); Paper: the authors' number (for PD-SORT, their released output); EXACT: Host within 0.2 HOTA of it; CLOSE: within 1.0. (b) The held-out UAVDT test split (20 drone videos, not used during design), ByteTrack with its published setting.

Metric	Paper	Host	Host +SCL	Change [95% CI]
(a) *PD-SORT* [31], *EXACT*				
HOTA↑	68.01	68.01	68.62	**+0.61 [+0.27, +1.65]**
MOTA↑	75.19	75.19	76.31	**+1.13 [+0.27, +3.03]**
IDF1↑	81.03	81.03	81.85	**+0.82 [+0.43, +1.94]**
Hybrid-SORT [32], *CLOSE*				
HOTA↑	67.1	66.70	66.70	0 (identical)
MOTA↑	75.8	75.52	75.52	0 (identical)
IDF1↑	78.0	77.56	77.56	0 (identical)
IDS↓	–	259	259	0 (identical)
(b) *ByteTrack, RT-DETR-L* [4, 16]				
MOTA↑		3.29	27.99	**+24.70 [+10.06, +51.08]**
IDF1↑		48.59	54.33	**+5.74 [+0.13, +13.11]**
IDS↓		1041	455	**−586 [−951, −248]**
ByteTrack, YOLOv8n [4, 15]				
HOTA↑		29.05	30.83	**+1.77 [+0.70, +2.94]**
IDF1↑		34.52	38.49	**+3.97 [+1.88, +6.17]**
IDS↓		719	393	**−326 [−691, −73]**

5.6 Generalization to Unseen Trackers

Table 13a gives the two predeclared external systems. PD-SORT [31] was reproduced exactly; with SCL, its HOTA rose from 68.01 to 68.62 (+0.61 points, $[+0.27, +1.65]$), its MOTA by 1.13 ($[+0.27, +3.03]$), and its IDF1 by 0.82 ($[+0.43, +1.94]$), with higher HOTA on 7 of 7 sequences. Like OC-SORT, it is a single-stage tracker, but it was not seen during design. Hybrid-SORT [32], a two-stage tracker, gave identical output, as predicted before the run.

5.7 Cross-Dataset Generalization on UAVDT

On the held-out UAVDT test split (Table 13b), SCL raised the HOTA of the YOLOv8n stream from 29.05 to 30.83 (+1.77 points, $[+0.70, +2.94]$) and its IDF1 by 3.97 points, and cut identity switches by 45%. With RT-DETR-L, whose host produced many false positives (MOTA 3.29), SCL raised MOTA from 3.29 to 27.99 (+24.70 points, $[+10.06, +51.08]$) and IDF1 by 5.74 points, and

cut false positives by 58% and identity switches by 56%.

6 Discussion and Practical Implications

The score scale is part of the MOT system. Halving the scores reduced ByteTrack and OC-SORT from 67.70 and 66.43 HOTA to 0, and with RT-DETR-L on UAVDT ByteTrack reached a MOTA of only 3.29. A tracker's fixed thresholds hold only for the detector they were tuned for, so the score scale deserves the same attention as the association method.

Deployment. SCL needs no training, no GPU, and no access to the inside of the detector or the tracker. On a CPU it takes 2.95 ms per frame, small next to the 42 ms of the detector, and the tracker itself ran faster with it (1.43 against 2.04 ms per frame); the end-to-end time rose by only 4.5% (4-vCPU Intel Xeon at 2.10 GHz, YOLOv8n and ByteTrack on three KITTI sequences, 2,279 timed frames). The label-free mode and confident share ρ can also be logged as a running check that a new or retrained detector still fits its tracker.

Wide applicability. One frozen configuration served seven trackers of both kinds, four detection sources, and three datasets of people and vehicles filmed from the ground, from a car, and from drones. The splits move exactly with a stretch or shift of the logits, and the layer also restored the trackers under cubed scores (Sect. 5.3), so it follows the common ways in which a detector's score scale changes.

7 Conclusion

This paper treated the compatibility between a detector's score scale and a tracker's thresholds as a property that can be checked and restored online, without labels and without changing either component. SCL checks it with nested Otsu splits of recent logits and a whole-stream majority rule, leaves compatible pairs as they are, and restores incompatible ones through the tracker's own published settings. The frozen layer was tested on seven trackers and four detection sources. Under halved scores, it restored two trackers from 0 to about 66 HOTA; with a mismatched detector on held-out aerial data, it raised MOTA by up to 24.70 points; well-matched trackers kept their own accuracy; and it took 2.95 ms per frame on average on a CPU. Detector–tracker compatibility can therefore be managed as part of the tracking system, so a detector can be replaced or retrained without retuning the tracker. Future work will extend the layer to end-to-end trackers, which learn detection and association jointly, and use the label-free mode and confident share as a running check when the detector of a deployed system is updated.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Adžemović, M.: Deep Learning-Based Multi-Object Tracking: A Comprehensive Survey from Foundations to State-of-the-Art. The Visual Computer **42**(9), 368 (2026) https://doi.org/10.1007/s00371-026-04607-7

[3] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple Online and Realtime Tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[4] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-Object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[5] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[6] Küppers, F., Kronenberger, J., Shantia,

A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https://doi.org/10.1109/cvprw50498.2020.00171

[7] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[8] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[9] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[10] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/electronics15174047

[11] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[12] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016). https://doi.org/10.48550/arXiv.1603.00831

[13] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021). https://doi.org/10.48550/arXiv.2107.08430

[14] Geiger, A., Lenz, P., Urtasun, R.: Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[15] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[16] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-Time Object Detection. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[17] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q.: The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 375–391. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01249-6_23

[18] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing, 1–10 (2008) https://doi.org/10.1155/2008/246309

[19] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[20] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[21] Efron, B., Hastie, T.: Computer Age Statistical Inference: Algorithms, Evidence, and Data Science. Cambridge University Press, Cambridge (2016). https://doi.org/10.1017/CBO9781316576533

[22] Li, Y., Liu, Y., Zhou, C., Xu, D., Tao, W.: A Lightweight Scheme of Deep Appearance Extraction for Robust Online Multi-Object Tracking. The Visual Computer **40**(3), 2049–2065 (2024) https://doi.org/10.1007/s00371-023-02901-2

[23] Bao, Y., Yu, Y., Qi, Y., Wang, Z.: Multiple Object Tracking with Adaptive Multi-Features Fusion and Improved Learnable Graph Matching. The Visual Computer **40**(4), 2279–2292 (2024) https://doi.org/10.1007/s00371-023-02916-9

[24] Zhang, Y., Liang, Y., Wang, J., Zhu, H., Wang, Z.: Enhanced Multi-Object Tracking via Embedded Graph Matching and Differentiable Sinkhorn Assignment: Addressing Challenges in Occlusion and Varying Object Appearances. The Visual Computer **41**(8), 6067–6085 (2025) https://doi.org/10.1007/s00371-024-03772-x

[25] Ding, Y., Wang, Y., Wang, L., Zhang, B., Li, J., Guo, L., Yang, Z., Liang, W.: Enhancing Multi-Object Tracking Efficiency through Result-Guided Feature Extraction and Query Filtering. The Visual Computer **41**(11), 8153–8167 (2025) https://doi.org/10.1007/s00371-025-03861-5

[26] Liang, Z., Xiao, G., Hu, J., Wang, J., Ding, C.: MotionTrack: Rethinking the Motion Cue for Multiple Object Tracking in USV Videos. The Visual Computer **40**(4), 2761–2773 (2024) https://doi.org/10.1007/s00371-023-02983-y

[27] Sun, Z., Lv, Z., Xiong, M., Hu, W., Wei, G., Zhou, J., He, T., Meng, Q.: Online Visual Multi-Object Tracking via Blind Super-Resolution and Clustering Learning. The Visual Computer **42**(4), 189 (2026) https://doi.org/10.1007/s00371-026-04382-5

[28] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022). https://doi.org/10.48550/arXiv.2206.14651

[29] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[30] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: Boosting the Similarity Measure and Detection Confidence for Improved Multiple Object Tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[31] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[32] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471

[33] Jung, H., Kang, S., Kim, T., Kim, H.: ConfTrack: Kalman Filter-Based Multi-Person Tracking by Utilizing Confidence Score of Detection Box. In: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 6569–6578 (2024). https://doi.org/10.1109/wacv57701.2024.00645

[34] Kim, S., Lee, S., Cho, M.: PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 5709–5716 (2026). https://doi.org/10.1609/aaai.v40i7.37491

[35] Goh, T.Y., Basah, S.N., Yazid, H., Aziz

Safar, M.J., Ahmad Saad, F.S.: Performance Analysis of Image Thresholding: Otsu Technique. Measurement **114**, 298–307 (2018) https://doi.org/10.1016/j.measurement.2017.09.052

[36] Tong, X., Ye, Z., Xu, Y., Gao, S., Xie, H., Du, Q., Liu, S., Xu, X., Liu, S., Luan, K., Stilla, U.: Image Registration With Fourier-Based Image Correlation: A Comprehensive Review of Developments and Applications. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing **12**(10), 4062–4081 (2019) https://doi.org/10.1109/jstars.2019.2937690